

# AIOS 项目任务书（专业详细版）

面向 Ubuntu 的本地优先 AI 原生操作系统原型

版本：v1.0 | 文档类型：任务书 / 技术执行蓝图 / 验收依据

|       |                                                          |
|-------|----------------------------------------------------------|
| 项目名称  | AIOS (AI-native Operating System for Ubuntu)             |
| 项目定位  | 作为独立启动项的 AI 原生操作系统，会话级提供自然语言、语音、多工具编排和安全执行能力             |
| 一句话目标 | 开机选择 AIOS 后，用户通过说话或打字即可安全完成打开/安装应用、文件处理、联网检索、旅行比价等系统任务   |
| 核心亮点  | 自然语言即系统 API；最小权限与二次确认；全链路审计与可撤销；本地模型优先；A/B 或 OSTree 原子更新 |

**执行摘要：** AIOS 不是“桌面加一个聊天框”，而是一套可从开机启动、系统会话、安全执行、工具注册、模型编排到审计回滚的完整 AI 原生操作系统原型。本任务书既用于研发落地，也用于团队分工、阶段验收和对外沟通。

# 1. 项目背景与建设目标

传统操作系统强调 GUI 和 CLI，人必须记住软件位置、命令格式、文件路径和跨应用操作流程。AiOS 的目标是把“语言”提升为系统级接口：用户不再先找应用再操作，而是先表达目标，由系统在安全边界内自动规划并执行。

本项目以 Ubuntu 22.04 LTS 为基座，通过 GRUB 独立启动、AI Shell 会话、SAL（System Action Language）动作语言、Tool Registry 声明式能力注册、执行代理、安全审计和可撤销机制，构建一套本地优先、可私有化、可扩展的 AI 原生操作系统原型。

## 1.1 总体目标

- 实现多启动：开机时在 GRUB 选择 Windows / Linux / AIOS，AiOS 作为独立启动项存在。
- 进入 AIOS 后默认进入 AI Shell 会话，支持 Super+K 命令面板、语音热词唤醒和权限预演弹窗。
- 以 SAL 作为唯一策略输出格式，确保策略模型不直接生成自由命令，避免不可控执行。
- 提供本地模型网关，统一编排 LLM、ASR、TTS、OCR、Embedding、Reranker 和可选 VLM。
- 提供首批高价值工具：打开/安装应用、打开网页、文件检索与处理、联网搜索问答、旅行机票酒店聚合比价。
- 形成最小权限、二次确认、审计、撤销、原子更新的安全闭环。

## 1.2 项目成功标准

| 维度     | 成功标准                         | 验证方式        |
|--------|------------------------------|-------------|
| 系统级可用性 | 可通过安装器部署并在 GRUB 中出现 AIOS 启动项 | 安装与重启测试     |
| 交互闭环   | 可通过语音/文字触发任务，看到预演、确认、执行和结果卡片 | E2E 演示用例    |
| 安全可控   | 高危动作强制确认；所有调用写入审计日志；文件动作可撤销  | 审计与回滚测试     |
| AI 能力  | 本地模型可完成意图解析、SAL 生成、检索问答和语音交互 | 模型健康检查与功能测试 |
| 商业化雏形  | 机票/酒店聚合、应用安装与网页打开等场景具备演示价值   | 业务场景 Demo   |

## 2. 建设范围与边界

### 2.1 本期纳入范围 (In Scope)

- 基于 Ubuntu 22.04 LTS 的 AIOS 安装镜像、GRUB 启动项与可恢复的系统更新机制。
- AI Shell 会话、命令面板、热词唤醒、权限预演、状态托盘、结果卡片与基础设置页。
- 策略模型只输出 SAL；执行代理根据 Tool Registry 完成渲染、预演、确认、执行、审计与撤销。
- 本地模型运行时：LLM、ASR、TTS、Embedding、OCR、Reranker，以及可选的 VLM 扩展位。
- 首发工具：apps.resolve/open/install、web.open\_url/search、files.search/open/move/delete、travel.search\_flights/hotels。
- 本地 RAG、会话记忆、偏好记忆、工具调用日志和资源级审计能力。

### 2.2 本期暂不纳入范围 (Out of Scope)

- 自动支付、自动下单、自动提交涉资或涉敏个人信息的交易动作。
- 无边界网页自动化、绕过官方客户端的高风险脚本化操作。
- 企业级多租户 SaaS 管理后台；本期以本地单机与私有化优先。
- 复杂视觉桌面代理的全自动操作；VLM 仅作为后续增强模块保留。

## 3. 总体架构设计

项目采用“Rust 负责做，Python 负责想，TypeScript/Tauri 负责看与控”的职责分离架构。策略、执行、安全和交互明确分层，确保模型能力提升不会破坏系统安全边界。

| 层级                | 职责                      | 技术栈                                  | 关键模块                                          |
|-------------------|-------------------------|--------------------------------------|-----------------------------------------------|
| Language Layer    | 将语音/文本输入转为结构化意图         | faster-whisper, openWakeWord, 规则+LLM | intent_parser, schema_guard, asr-service      |
| Interaction Layer | 展示命令面板、权限预演、状态与结果       | React, Tauri, GTK/Wayland 集成         | command-palette, status-panel, settings-ui    |
| Planning Layer    | 生成 SAL/任务图/DAG，插入确认节点   | Python, langgraph/自研状态机              | planner-service, model-router                 |
| Execution Layer   | 工具解析、dry-run、权限校验、执行、回滚 | Rust                                 | tool-executor, policy-engine, rollback-engine |

|                           |                                 |                                         |                                                    |
|---------------------------|---------------------------------|-----------------------------------------|----------------------------------------------------|
| Memory & RAG              | 本地知识库、会话记忆、偏好记忆和混合检索            | bge-m3, SQLite-vec/FAISS, BM25          | rag-service, memory-service, indexer-service       |
| Model Runtime             | 提供 LLM/ASR/TTS/OCR 等统一服务        | llama.cpp, vLLM, Piper, Tesseract       | llm-service, asr-service, tts-service, ocr-service |
| OS Integration & Security | 会话、systemd、AppArmor、Polkit、原子更新 | systemd, AppArmor, Polkit, Btrfs/OSTree | agentd, auditd, infra/*                            |

3.1 核心数据流

语音或文字输入首先进入 Language Layer，经由 ASR、文本净化和意图解析后形成结构化请求；Planning Layer 根据上下文生成 SAL；Execution Layer 根据 Tool Registry 渲染执行计划并进行权限预演；用户确认后，Rust 执行器在沙箱与最小权限控制下完成动作；随后审计与撤销记录回写到本地数据库和日志流中，最终 UI 展示执行状态与结果。

4. 启动、安装与系统会话

4.1 启动与安装

- 采用 GRUB 作为统一引导器，AIOS 与 Windows、Linux 并列作为独立启动项。
- 安装器建议选用 Calamares（图形）或 Subiquity（文本），支持一键安装。
- 文件系统建议采用 Btrfs 子卷 + /home 独立，支持快照和恢复；更新方案可选 A/B 分区或 OSTree。
- 支持 LUKS2 全盘加密，可选 TPM 绑定与安全启动配套配置。

4.2 AI 会话与守护进程

- 显示栈基于 Wayland + GNOME/KDE；AI Shell 作为会话级系统壳承载主要交互。
- systemd 管理 aios-supervisor，负责拉起模型网关、Planner、RAG、Tool Registry、审计和执行代理。
- agentd 作为本地控制平面守护进程，提供状态流、任务调度和服务健康检查。

## 5. SAL 与 Tool Registry 设计原则

SAL (System Action Language) 是策略模型唯一允许输出的动作语言。模型不直接返回 shell 命令，而是只返回版本号、任务目标、节点和边；执行代理根据工具注册表把每个节点翻译成安全、可审计、可预演的系统动作。

### 5.1 SAL 设计原则

- 仅允许结构化 JSON 输出，字段固定且可校验。
- 节点必须引用已注册工具；禁止任意未注册动作。
- 支持节点之间通过引用传值，例如 @resolve.app\_id。
- 支持 DAG，便于多步任务、并行查询与确认门插入。

### 5.2 Tool Registry 设计原则

- 所有工具必须声明参数 Schema、权限需求、危险等级、是否支持 dry-run、是否支持 rollback。
- 工具实现方式可以是 Rust 内置、Python 入口或受控 exec 模板，但均需经过统一执行器。
- 权限声明至少覆盖 process\_spawn、fs\_read、fs\_write、net\_out、clipboard 等资源域。
- 每个工具都要提供 UI 提示模板，用于权限预演和确认弹窗的可解释展示。

## 6. 功能需求

| 编号    | 需求               | 说明                                                     |
|-------|------------------|--------------------------------------------------------|
| FR-01 | 支持通过文字与语音发起系统任务。 | 用户可以在命令面板中输入文本，也可以通过热词唤醒后使用语音下达指令。                     |
| FR-02 | 支持本地应用与游戏解析和打开。  | 系统需识别 .desktop、Flatpak、Steam、Lutris 等来源并给出正确 launcher。 |
| FR-03 | 支持应用安装。          | 系统需通过 Flatpak/Steam/Lutris 等受控途径安装应用或游戏，并要求中风险确认。      |
| FR-04 | 支持文件检索和打开。       | 系统需对 Desktop、Documents、Downloads 等目录建立索引，并支持时间、        |

|       |                |                                     |
|-------|----------------|-------------------------------------|
|       |                | MIME 类型等过滤。                         |
| FR-05 | 支持网页打开和联网检索。   | 系统需在白名单域名范围内执行搜索与问答，并展示来源链接。        |
| FR-06 | 支持旅行机票与酒店聚合搜索。 | 系统需并发请求多个 provider，完成字段标准化、去重和排序。   |
| FR-07 | 支持权限预演与二次确认。   | 高风险与中风险动作必须在执行前展示命令、资源触达和风险等级。      |
| FR-08 | 支持审计与撤销。       | 所有工具调用必须记录日志；文件删除、移动、重命名与覆盖动作必须可撤销。 |

7. 非功能需求

| 类别   | 要求                  | 目标值/原则            | 验证方式      |
|------|---------------------|-------------------|-----------|
| 安全   | 默认最小权限、白名单出网、高危二次确认 | 拒绝未注册工具和未声明资源     | 策略和执行器联测  |
| 可靠性  | 失败可回滚、服务可重启、更新可恢复   | 关键服务 systemd 自动拉起 | 故障注入与重启测试 |
| 性能   | 短任务低延迟返回            | 面板交互亚秒级反馈，模型按路由降级 | 性能测试      |
| 隐私   | 本地模型优先，云推理需显式授权     | 不默认上传用户文件与语音      | 配置审计      |
| 可扩展性 | 支持第三方工具与 SDK        | 新增工具不改动核心执行器      | 插件接入测试    |

8. 安全、审计与合规要求

- 执行器必须位于系统安全边界之内。Planner 不能直接访问文件系统、进程或网络。
- 所有高危工具默认需要二次确认，例如 files.delete、apps.install、upload、非白名单 net.request。

- 使用 AppArmor、seccomp、bubblewrap 或 Flatpak 沙箱限制进程边界。
- 审计日志至少记录：时间戳、工具、参数、命令、退出码、stdout/stderr 摘要、资源触达范围、undo 钩子。
- 隐私原则：本地模型优先；调用云模型或外部 API 时必须显式配置并在 UI 说明用途。

9. 模型与运行时策略

模型层采用分工明确的路由方式：短意图解析与参数补全优先走 7B 量化模型；复杂规划与长上下文检索问答可路由到 14B 模型或 GPU 服务；语音识别使用 faster-whisper，语音播报使用 Piper，文档语义检索使用 bge-m3，OCR 使用 Tesseract 或后续更强替代。

| 能力域       | 首选模型/引擎                          | 部署方式             | 作用              |
|-----------|----------------------------------|------------------|-----------------|
| LLM-轻量    | Qwen2.5-7B / Llama 3.1 8B        | llama.cpp / vLLM | 意图解析、参数补全、短规划   |
| LLM-中型    | Qwen2.5-14B                      | vLLM / GPU       | 复杂 DAG、长上下文 RAG |
| ASR       | faster-whisper medium / large-v3 | 本地服务             | 流式语音识别          |
| TTS       | Piper                            | 本地服务             | 语音播报            |
| Embedding | bge-m3 / e5-mistral              | 本地服务             | 文档索引、检索、记忆      |
| OCR       | Tesseract / Surya-ocr (后续)       | 本地服务             | 图片/PDF 文本提取     |

10. 研发里程碑与阶段任务

| 阶段 | 周期      | 目标             | 核心任务                                              | 阶段交付                 |
|----|---------|----------------|---------------------------------------------------|----------------------|
| P0 | 第 1-4 周 | 形成可引导与最小闭环     | 安装器、GRUB、AI Shell、ASR/TTS、apps.resolve/open、审计基础版 | 可演示 ISO + “打开应用” E2E |
| P1 | 第 5-8 周 | 完成执行闭环与文件/网页能力 | Tool Registry、执行器、files.*、web.open_url/se         | 文件检索与网页打开 E2E        |

|    |           |             |                                            |              |
|----|-----------|-------------|--------------------------------------------|--------------|
|    |           |             | arch、RAG、权限预演、撤销                           |              |
| P2 | 第 9-12 周  | 引入旅行聚合与 SDK | travel.search_flights/hotels、第三方工具接入、开发者文档 | 可展示机票/酒店聚合结果 |
| P3 | 第 13-16 周 | 性能优化与更新恢复   | 模型路由优化、A/B/OSTree、可视化 workflow、监控与恢复       | 稳定演示版与测试报告   |

11. 交付物与验收标准

| 交付物   | 内容                                                   | 验收标准                |
|-------|------------------------------------------------------|---------------------|
| 系统原型  | AIOS 镜像、安装器、GRUB 启动项、AI Shell                        | 可以安装、启动、进入 AI 会话    |
| 控制平面  | aios-supervisor、agentd、planner-service、tool-executor | 关键服务可拉起并完成任务链路      |
| 工具集   | 应用、网页、文件、旅行聚合工具                                      | 工具定义完整，权限与 UI 提示正确  |
| 模型服务  | LLM、ASR、TTS、Embedding、OCR                            | 服务可独立健康检查并被路由调用     |
| 安全与审计 | 权限策略、日志、撤销、恢复方案                                      | 高危动作需确认，审计可查询，文件可恢复 |
| 文档资料  | README、开发文档、运维文档、测试报告、演示视频                           | 支持团队接力开发与外部展示       |

12. 团队角色建议

| 角色      | 人数建议 | 职责            |
|---------|------|---------------|
| 产品/项目经理 | 1    | 需求收敛、版本规划、验收组 |



|             |   |                                              |
|-------------|---|----------------------------------------------|
|             |   | 织、对外沟通                                       |
| 系统/安全后端     | 2 | agentd、tool-executor、<br>policy-engine、审计与沙箱 |
| AI/模型工程     | 2 | 模型路由、SAL 策略、RAG、<br>ASR/TTS、Prompt 与评测       |
| 客户端/UI      | 2 | 命令面板、权限弹窗、结果卡<br>片、设置页、托盘                    |
| DevOps/基础设施 | 1 | 安装器、构建、CI/CD、镜像、<br>监控、更新恢复                  |

13. 主要风险与应对

| 风险               | 影响          | 应对策略                                                       | 优先级 |
|------------------|-------------|------------------------------------------------------------|-----|
| 策略模型幻觉或错误规划      | 可能触发错误工具或参数 | 强制 JSON Schema、<br>仅允许已注册工具、<br>Planner 与 Executor<br>完全解耦 | 高   |
| 权限模型过宽           | 带来安全与合规风险   | 最小权限设计、白名单<br>出网、会话级 token 和<br>二次确认                       | 高   |
| 模型资源不足           | 体验下降或失败率高   | 模型路由降级、CPU/<br>GPU 双栈、缓存与异步<br>预热                          | 中   |
| 第三方 API 限流或价格不一致 | 旅行聚合体验不稳定   | 多 provider 并发、缓<br>存、结果标注来源与更<br>新时间                       | 中   |
| 安装器与更新恢复复杂度高     | 影响系统级交付质量   | 先做单机稳定闭环，再<br>逐步补齐 A/B 或<br>OSTree                         | 中   |

## 14. MVP 立即启动清单

1. 完成 Calamares 安装器配置与 AIOS 启动项预置。
2. 实现 aios-supervisor，拉起模型网关、Planner、执行代理和审计服务。
3. 接入 faster-whisper + openWakeWord + Piper，完成语音最小闭环。
4. 落地 SAL Schema 与首批 Tool Registry：apps.open/install、web.open\_url、files.search。
5. 实现 apps.resolve：扫描 .desktop、Steam、Lutris、Flatpak 并建立索引。
6. 实现 tool-executor：参数校验、权限预演、执行、日志、撤销。
7. 完成 AI Shell 命令面板、权限弹窗与结果卡片。
8. 打通本地 RAG：bge-m3 + SQLite-vec/BM25。
9. 搭建 travel.search\_flights/hotels 的接口骨架与并发聚合逻辑。
10. 补齐 A/B 或 OSTree 原子更新与 Recovery 启动项设计。

## 15. 本次补充交付说明

已有上传文件包含完整项目骨架压缩包与 README。为了便于团队协作和对外沟通，本次补充交付新增以下文档：

- 《AIOS 项目任务书（专业详细版）》DOCX：适合继续编辑、内部评审和对外发送。
- 《AIOS 项目任务书（专业详细版）》PDF：适合固定版式传阅。
- 《AIOS 项目任务书（专业详细版）》Markdown：适合直接放入 GitHub /docs 目录。
- 文档打包 ZIP：便于一次性下载所有补充资料。